BrunoRAG

Eric Xia, Angel Alvarado Reyes, Julian Dai, Isaac Calderon, Derek Truong

Brown University

abstract

As avid users of Courses@Brown, we wanted to investigate information retrieval systems which allow users to search for courses through natural language. With recent developments, LLM-based interactions can be made more resilient, informative, and enjoyable than ever before.

We present BrunoRAG: a search service which allows users to query for courses using natural language, which uses vector embeddings to match with the courses database. Using a distributed model allows nodes to distribute both data preprocessing steps (such as crawling and indexing) and live query operations. The result is a fast and effective chat service which allows users to find courses through nearly any factor: interests, timeslots, and even workload, special topics, and exam formats.

data

We use the Courses@Brown (CAB) API to collect ~131,000 course section records spanning Summer 2016 through Spring 2026, representing ~10,000 unique offerings. We also use Critical Review data from 2019 to 2025.

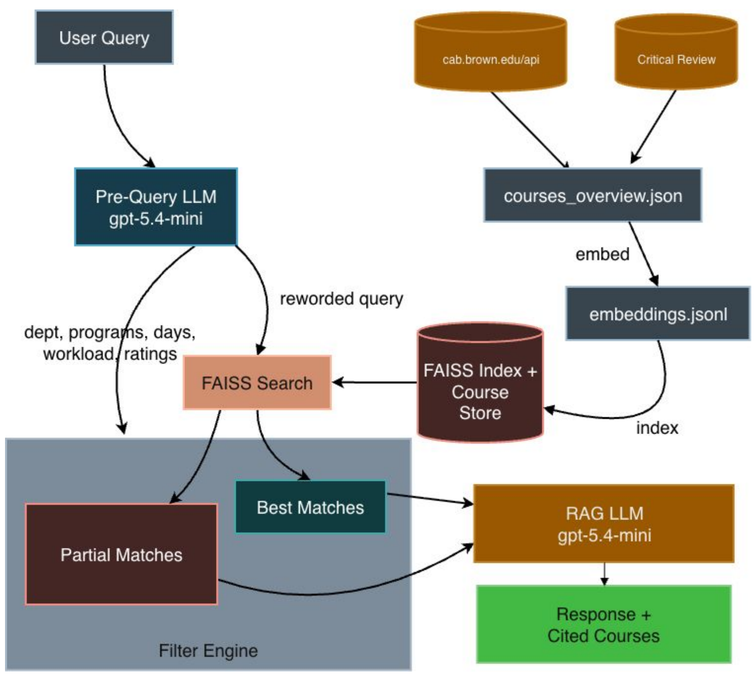
A two-step pipeline queries a search endpoint for listings, then fetches a details endpoint per CRN for descriptions and metadata. Critical Review data, when available, is joined to the dataset via CRN, which the API already exposes per section.

Supported Queries

semantics in philosophy ✓
english classes <10 hours/wk MWF ✓
engineering class with hands-on component ✓
TuTh systems-heavy CS courses ✓
Linguistics courses with paper writing ✓
Cities and urban architecture practicum ✓
History of music ✓
intensive drama courses ✓

One problem we faced was how to handle filter requests systematically, which is crucial to ensure the service maintains a natural quality. By performing both query augmentation and filtering prior to FAISS, our approach is able to uniquely handle a wide range of requests in a reasonable and helpful manner.

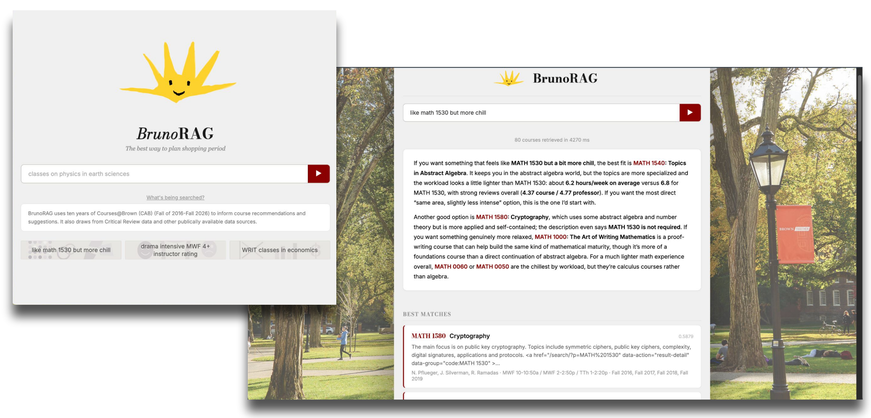
methodology



At seed time, course data is crawled, converted to structured text, embedded with metadata such as meeting days, times, and professor, and distributed across N nodes using consistent hashing. At query time, the user's input is embedded through the OpenAI API to capture semantic similarity to course data. A pre-query model extracts keyword features such as days of the week and professor which are applied programmatically.

The query embedding and extracted filters are sent to N nodes to retrieve local top-K matches. After filtering, each node uses FAISS to index relevant embeddings and return the most similar results; the coordinator then merges these into a global top-K set, which is passed to the post-query model as context for the final response. A coordinator-layer LRU cache serves repeated queries, bypassing the embedding and retrieval pipeline.

UI



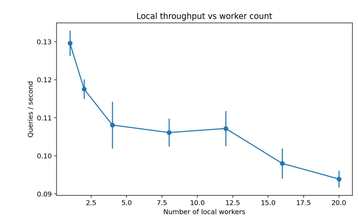
next steps

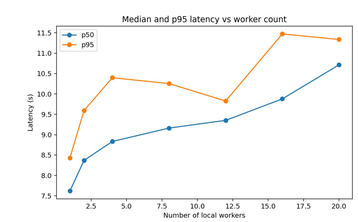
- Reduce network overhead by minimizing query fanout
- Add user personalization profiles such including major, interests, and course history
- Context-aware caching to serve semantically similar queries
- Deploy with containerized microservices such as Docker + Kubernetes for horizontal scaling
- Support multi-turn conversations with memory

Benchmarking

Our crawl step takes about two hours to complete and is restricted by API limits rather than processing constraints. Per-node indexing takes 10-15 seconds, while queries average 3-4 seconds on average.

Our distributed setup (n=4) did not improve the throughput or latency.





references

Dean, J. and Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. Communications of the ACM, 51(1):107–113.

Lehman, E., Hernandez, E., Mahajan, D., Wulff, J., Smith, M. J., Ziegler, Z., Nadler, D., Szolovits, P., Johnson, A., and Alsentzer, E. (2023). Do we still need clinical language models? Proceedings of the Conference on Health, Inference, and Learning, 209:578–597.

acknowledgements

We would like to thank Jeffrey Pogue & Uriel Cohen Priva for the C@B data. We would also like to thank our 1380 mentor Ethan Lavi for thoughtful review of our initial poster.

Create alignment guides and use them consistently—the eye will find even small misalignments. Large text may need to be nudged.

Best titles are short conclusions, not long introductions.

Martin Krzywinski mkweb.bcgsc.ca/poster.design

Canada's Michael Smith Genome Sciences Center bcgsc.ca

Avoid long addressess—no postal codes, no zip codes.

Your work is a "study" and explores a "relationship" to look for an "effect". Treat that as a given and say what is important.

Don't try to be snarky, cheeky or witty—most attempts do not succeed. Don't trigger the jokers, cynics, cranks and curmudgeons.

Don't say everything you know—make room for empty space. Your most valuable resource is the reader's time.

Balance visual weight and size the logos equally. If acknowledging institutional support, place it next to the logo.

Use vector-based logos, not low-resolution bitmaps. Do not change logos' aspect ratio or crowd it with other elements—both are likely against its branding style.

Regions of unbalanced negative space are good candidates for annotations, credits, quotes, and other garnish that adds value to the poster. Don't overdo it—most quotes rehash old tropes. If you must, find something that is passionate and slightly mysterious (http://mkweb.bcgsc.ca/quotes).

bcgsc.ca bccancer.ca

POSTER CHILD OF SCIENCE

A poster is your first opportunity to organize and communicate your reasearch to members outside of your lab. It will help you to practise telling and "drawing" your science story and its design should be based on its concepts, themes and transitions.

Most posters are bad not because they are ugly (they are) but because they fail to present concisely what was done and, more importantly, why it was done. Most posters have too much on them. Less is more: get to the point, then stop.

Map salience to pertinence. When used in moderation, colors like orange or magenta say "look here". You cannot look everywhere.

ESTABLISH A STORY PATH and stick to it. Saliently and intuitively code key contrasts (e.g. healthy/disease, wildtype/mutant).

The reader won't know what is important, so tell them.

USE COLOR FOR THEMES OR DATA ENCODING and not as garnish. The first color to appear should begin the core story. Avoid unintentional emphasis by equalizing for perceptual luminance.

Round corners slightly for eye comfort.

Extend beyond the frame to imply a crop or continuity.

Good explanations are ones conveniently placed. Embed simple diagrams next to relevant text. Some graphics don't need a legend—make room for explanations within. Callout lines should be rectilinear or at 45° if the graphic already has such elements.

ALL SCIENCE DESERVES EXCELLENT EXPLANATIONS

Explain quickly and clearly. Motivate why the work was done—what is the cost of not doing it?

The poster is your prop. In most settings, you will be there to present it. Match its content to the story you will tell.

ONLY YOU CAN STOP POSTER DUMPSTER FIRES

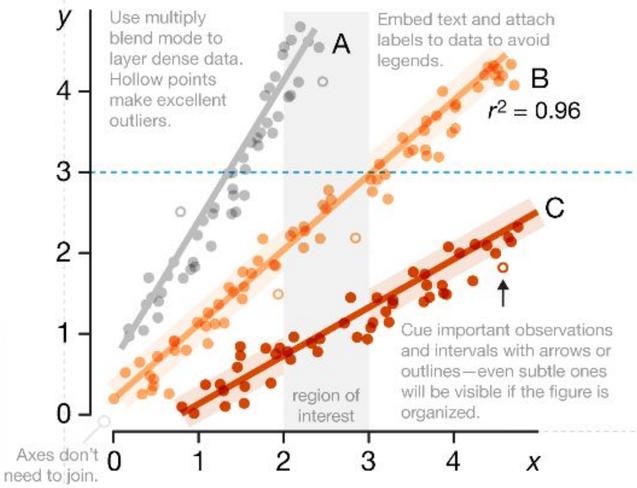
Colored text may help emphasize a theme but use it sparsely.

Clip art, pie charts, bullet points, boxes around text, background fills and gradients. Only you can stop it.

Maintain good Gestalt—similar shapes and colors will form groups. Use them to encode real-world relationships and be on the lookout for unintended accidental groupings.

1 Use figure titles to explain trends, not merely to specify the axes.

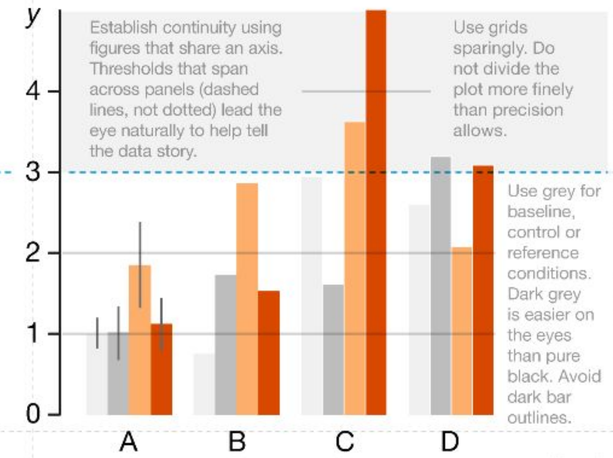
Establish subordinate content with italics.



▲ Don't tell the reader what is obvious: "a linear fit to a scatter plot" is redundant. Don't tell me what I'm seeing—interpret the figure instead. Italicize variables in fit diagnostics and use shaded bands for confidence intervals. Highlight regions of interests with a solid color (or grey), not outlines.

2 Share axes or align panels to clarify variables or emphasize changing scale.

Reserve small text for tangents and detail beyond the first explanation.

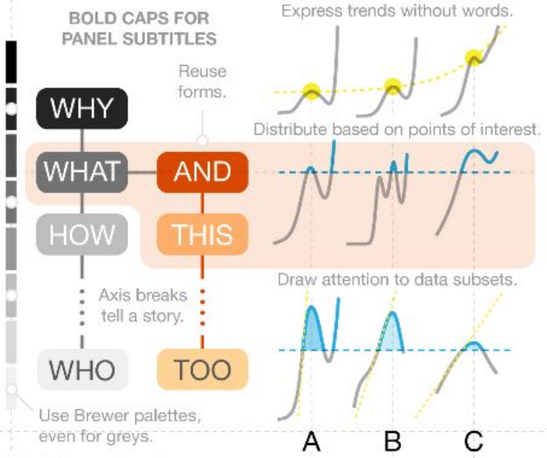


▲ Categorical variables in bar charts do not need an explicit axis. Specify sample sizes and what error bars represent (e.g. standard error of mean, $n = 5$). Report P-values with effect sizes or confidence intervals. A statistically significant observation isn't necessarily of biological interest.

Alignment of similar quantities subtly suggests where to look next.

3 Reveal qualitative trends in small multiples with order, cutoffs and cues.

Use a lead to announce an observation or insightful comment.



▲ Look for opportunities to include key observations (●) and explanations in the figure—don't leave it to the main text, where it may be far from the graphic. Emphasize what quantities are important—anticipate the reader's questions and answer them.

Use typographical garnish sparingly—be creative, but in small steps. A well-placed symbol or label can connect themes or indicate the purpose of text (e.g. triangles suggest a legend).

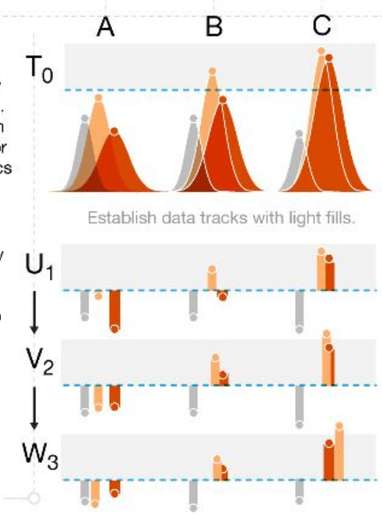
Allow content to establish layout proportions and do not emphasize gutters with lines. One or two such dividers can be effective, but too many will turn the poster into a jail.

USE SMALL MULTIPLES

Use ink sparingly to make compact figures legible—dense is not necessarily crowded. Explain an encoding once and reuse it. Create a visual key for complex encodings and choose graphical explanations over text.

Avoid visual complications that are not relevant—for example, color blending can create distracting intersections of color. Superimpose white outlines to emphasize shapes with an opaque fill.

TABULATE plots and text seamlessly with a column or row for explanations. Italicize text with care and look for unintended italics in subscripts.



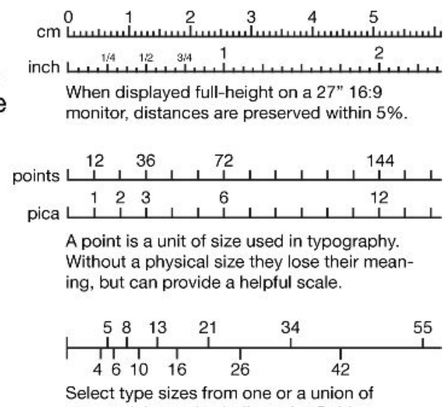
ARROWS imply a relationship or change. Do not use them to guide the eye, which can be achieved with spacing and alignment.

MAINTAIN AND CONTROL PROPORTIONS

This poster is 16" × 12" (1152 × 864 pt), uses Helvetica Neue with a 5, 8, 13, 21, 34, 55 pt scale ladder and is legible on most screens.

Sans-serif is clearer than serif at small sizes and suitable for modest amounts of copy.

Keep line length short and hyphenate instead of fully justifying.



EVERYTHING IS IMPORTANT, BUT SOME THINGS ARE MORE IMPORTANT THAN OTHERS. Establish a visual hierarchy by emphasizing your hypothesis, conclusion and the key points that connect them. Relegate protocols, technical methods, and other minutiae to the bottom of the poster. Always be mindful of what the reader needs to know to understand enough to ask insightful questions and frontload this information.

FORCE LINE BREAKS that split a sentence into noun phrases or offer a natural pause, such as at a comma or a period. Balance layout by shortening sentences—there are many ways to say something and some ways are easier to typeset. Do not let a template bully you into using a specific column width. Change proportions to suit content.

AVOID OBVIOUS HEADINGS such as "references". Citations can be set in a block of text, with bold numbers like this **1.** R. Bringhurst, *Elements of Typographical Style.* 4th ed (2012) and **2.** W. Strunk Jr., *Elements of Style* (1918). Unless a specific citation style is required, use a compact style that also includes the title.

planning

- (Eric)

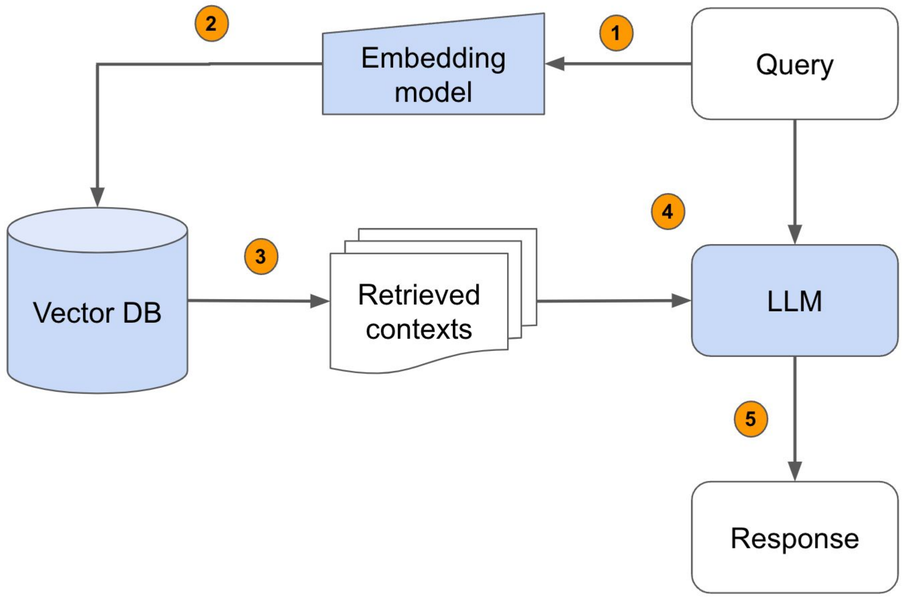
Data pushed with GIT LFS ✓

work on the actual distribution code from Isaac ✓

Try to get distribution working on CAB locally by this **Wednesday 8th.** ✓

Critical Review stuff. Ideal world is distributing the frontend.

- We need someone to handle deployment to the cloud / setting up cloud development. Can deploy distributed version with test sandbox (Isaac). Goal: by **Wednesday 8th**



- (Angel, Derek) figure out RAG details, and database. Initial plan by **Wednesday 8th.**

(Julian)

Draw plan for query distribution: distributed key-value. Embed with pinecone, with metadata, (vector of description + metadata)

rag

1. 1. Seed data

   Assumption: data stored in static file (data.json)

   Script hashes by course code (consistent / naive hashing), stores data in one of N nodes

   Each data stores it locally (in a file)

2. 2. Query time (Search API)

   The distributed system's job is to serve as a search API — input = query, output = relevant data from CAB

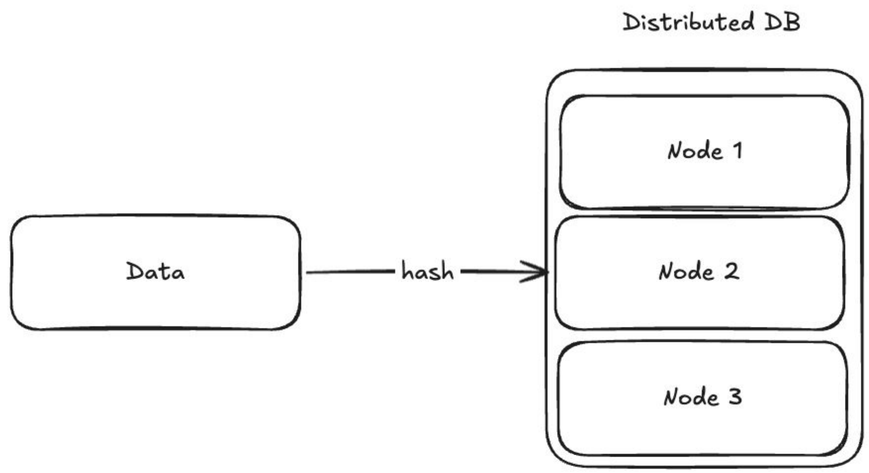
   Query gets sent to all nodes. Each node runs cosine similarity / key word search (depending on what the agent decides) between the query + all records that node has on file (outputting a numerical score)

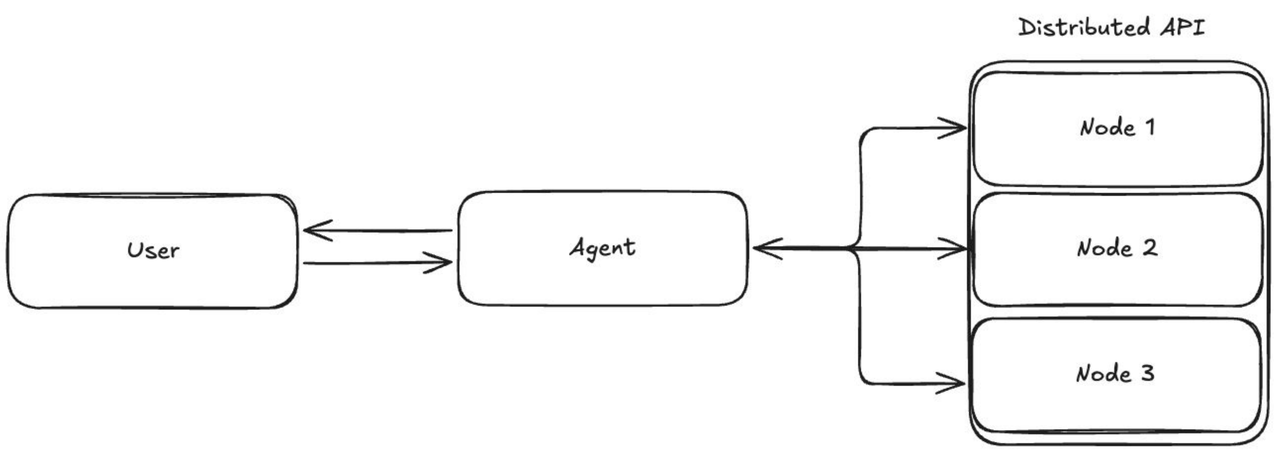
   We then combine all results, filtering to top_k results (by score) and return to user.
   LLM takes returned context and responds

What to do next

Deploying current: eric, isaac
Note: baseline for our RAG should be Pareto-efficiency on cost + latency for giving an LLM all the courses

Stage	Current implemented search	Ideal RAG system
Crawl	crawl() retrieves data in parallel (MR, optional), then data is groups.put() across all nodes (distributed) ✓	crawl() retrieves data in parallel (MR, optional), then data is groups.put() across all nodes (distributed) ✓
Index	embed() embeds each value (course) into the database (MR) ✓	embed() embeds each value (course) into the database (MR)
Query	(MR) compute TF-IDF from query keywords or cosine similarity between QV ✓	(MR) compute cosine similarity between query and values (courses) and return top K
deploy	Not deployed	Deployed (Isaac)
Agent / Retrieval Layer	None	Agent loop sits on top of retrieval and is not distributed. Agent functionality uses top K for retrieving relevant data on courses.
Structured Filtering	None	Agent loop can handle structured lookups, such as MWF-course preferences, by accessing original values and metadata (optional)
Output / Citations	Returns top K results directly from course information	LLM can answer wide range of course related questions. Agent loop can return verifiable citations with structured course information (optional)
benchmark	Not benchmarked	Benchmarked with improvements over single-node instance


Distributed DB
Data
hash
Node 1
Node 2
Node 3


Distributed API
User
Agent
Node 1
Node 2
Node 3